

Neural Networks (PyTorch) — Project Recipe Card

9-step recipe · training loop · loss/optimizer table · regularisation · debug checklist · persistence. Deep companion: neural_networks_cheatsheet.md + neural_networks.ipynb

◆ 9-step recipe

- 1. **Setup** — seeds, device, deterministic flags
- 2. **Split** — train / val / test (3 sets, NOT 2)
- 3. **Scale** — fit scaler on TRAIN only; transform val/test
- 4. **Tensors + DataLoader** — shuffle TRAIN only; batch size powers of 2
- 5. **Model** — nn.Module subclass or nn.Sequential
- 6. **Loss + Optimizer** — match to task; Adam @ 1e-3 default
- 7. **Training loop** — zero_grad → forward → loss → backward → step
- 8. **Evaluate on TEST** — once, after tuning is done
- 9. **Diagnose** — loss curves, regularise, refit

Universal contract: output layer + dtype + loss MUST agree. Mismatch = silent bug (NaN loss, near-random accuracy, exploding gradients).

◆ Starter skeleton · A: setup + data

```
import numpy as np, random, torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. SETUP — reproducibility
SEED = 42
np.random.seed(SEED); torch.manual_seed(SEED);
torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True;
torch.backends.cudnn.benchmark = False
# Full determinism (small speed cost):
# torch.use_deterministic_algorithms(True) # also set
CUBLAS_WORKSPACE_CONFIG=:4096:8
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("device:", device)

# 2. SPLIT (3 sets)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                         random_state=SEED,
                                         stratify=y if
classification else None)
X_tr, X_va, y_tr, y_va = train_test_split(X_tr, y_tr,
                                         test_size=0.2, random_state=SEED,
                                         stratify=y_tr if
classification else None)

# 3. SCALE — fit on TRAIN only (avoid leakage)
sc = StandardScaler().fit(X_tr)
X_tr_s, X_va_s, X_te_s = sc.transform(X_tr), sc.transform(X_va),
sc.transform(X_te)

# 4. TENSORS + DATALOADER
to_t = lambda a, dt=torch.float32: torch.tensor(a, dtype=dt)
y_dt = torch.long if multiclass else torch.float32
dl_tr = DataLoader(TensorDataset(to_t(X_tr_s), to_t(y_tr,
y_dt)),
                  batch_size=64, shuffle=True,
                  drop_last=True, # avoid
BN(batch=1) crash
                  num_workers=2,
pin_memory=(device.type=="cuda"))
dl_va = DataLoader(TensorDataset(to_t(X_va_s), to_t(y_va,
y_dt)),
                  batch_size=256, shuffle=False,
                  num_workers=2,
pin_memory=(device.type=="cuda"))
dl_te = DataLoader(TensorDataset(to_t(X_te_s), to_t(y_te,
y_dt)),
                  batch_size=256, shuffle=False)
```

◆ Starter skeleton · B: model + train loop

```
# 5. MODEL (MLP)
class MLP(nn.Module):
    def __init__(self, in_dim, hidden=64, out_dim=1, p=0.2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden), nn.ReLU(), nn.Dropout(p),
            nn.Linear(hidden, hidden), nn.ReLU(), nn.Dropout(p),
            nn.Linear(hidden, out_dim))
    def forward(self, x): return self.net(x)

model = MLP(X_tr.shape[1], hidden=64, out_dim=1).to(device)

# 6. LOSS + OPTIMIZER (regression example)
loss_fn = nn.MSELoss()
opt = torch.optim.Adam(model.parameters(), lr=1e-3,
weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(opt,
patience=5, factor=0.5)

# 7. TRAIN LOOP with early stopping
best_val, patience, bad = float("inf"), 10, 0
hist = {"tr": [], "va": []}
for epoch in range(200):
    model.train()
    tr_loss = 0.0
    for xb, yb in dl_tr:
        xb, yb = xb.to(device), yb.to(device)
        opt.zero_grad()
        pred = model(xb).squeeze(-1)
        loss = loss_fn(pred, yb)
        loss.backward()
        opt.step()
        tr_loss += loss.item() * xb.size(0)
    tr_loss /= len(dl_tr.dataset)

    model.eval()
    va_loss = 0.0
    with torch.no_grad():
        for xb, yb in dl_va:
            xb, yb = xb.to(device), yb.to(device)
            pred = model(xb).squeeze(-1)
            va_loss += loss_fn(pred, yb).item() * xb.size(0)
        va_loss /= len(dl_va.dataset)
    scheduler.step(va_loss)
    hist["tr"].append(tr_loss); hist["va"].append(va_loss)

    if va_loss < best_val - 1e-4:
        best_val, bad = va_loss, 0
        torch.save(model.state_dict(), "best.pt")
    else:
        bad += 1
        if bad >= patience:
            print(f"early stop at epoch {epoch}, best
val={best_val:.4f}")
            break

# 8. EVAL on TEST — ONCE
model.load_state_dict(torch.load("best.pt"))
model.eval()
with torch.no_grad():
    preds = torch.cat([model(xb.to(device)).cpu() for xb, _ in
dl_te]).squeeze(-1).numpy()
# Compute metrics in sklearn (RMSE, R², ROC-AUC, F1, ...)
```

● Output layer · loss · dtype · metrics

Task	Output layer	Loss	y dtype	Metrics
Regression	nn.Linear(h, 1)	nn.MSELoss (or L1Loss, HuberLoss)	float32	RMSE, R²
Binary cls	nn.Linear(h, 1) (LOGITS)	nn.BCEWithLogitsLoss	float32 (0/1)	Acc, ROC-AUC, F1
Multi-class	nn.Linear(h, K) (LOGITS)	nn.CrossEntropyLoss	long (class idx)	Acc, macro-F1
Multi-label	nn.Linear(h, K) (LOGITS)	nn.BCEWithLogitsLoss	float32 (0/1 per col)	per-label F1
Counts	nn.Linear(h, 1) + softplus	nn.PoissonNLLLoss	float32	RMSE
Quantile	nn.Linear(h, Q)	pinball (custom)	float32	quantile loss

Double-sigmoid trap: sigmoid + BCELoss = correct but numerically unstable. **NO sigmoid + BCEWithLogitsLoss** = correct AND stable. Same for softmax + NLLLoss vs raw logits + CrossEntropyLoss.

▲ Architecture patterns

Tabular MLP (the default)

```
class MLP(nn.Module):
    def __init__(self, in_dim, hidden=(128, 64), out_dim=1,
p=0.2, bn=False):
        super().__init__()
        layers = []
        prev = in_dim
        for h in hidden:
            layers += [nn.Linear(prev, h)]
            if bn: layers += [nn.BatchNorm1d(h)]
            layers += [nn.ReLU(inplace=True), nn.Dropout(p)]
            prev = h
        layers += [nn.Linear(prev, out_dim)]
        self.net = nn.Sequential(*layers)
    def forward(self, x): return self.net(x)
```

2-3 hidden layers, hidden width 64-256, ReLU, Dropout 0.1-0.3, optional BN.

Embeddings for categorical

```
class TabularNN(nn.Module):
    def __init__(self, cat_cardinalities, n_cont, hidden=128,
out_dim=1):
        super().__init__()
        # Embedding dim heuristic: min(50, (k+1)//2)
        self.embs = nn.ModuleList([
            nn.Embedding(k, min(50, (k+1)//2)) for k in
cat_cardinalities])
        emb_dim = sum(e.embedding_dim for e in self.embs)
        self.bn = nn.BatchNorm1d(n_cont)
        self.head = nn.Sequential(
            nn.Linear(emb_dim + n_cont, hidden), nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(hidden, out_dim))
    def forward(self, x_cat, x_cont):
        z = torch.cat([e(x_cat[:, i]) for i, e in
enumerate(self.embs)], dim=1)
        return self.head(torch.cat([z, self.bn(x_cont)], dim=1))
```

CNN (images)

```
# Use a pretrained backbone – almost never train from scratch
import torchvision.models as M
weights = M.ResNet50_Weights.DEFAULT
model = M.resnet50(weights=weights)
preprocess = weights.transforms() # the right
input transforms FREE
# Replace head:
model.fc = nn.Linear(model.fc.in_features, num_classes)
# Freeze backbone for transfer learning:
for p in model.parameters(): p.requires_grad = False
for p in model.fc.parameters(): p.requires_grad = True
```

LSTM / GRU (sequence)

```
class SeqModel(nn.Module):
    def __init__(self, in_dim, hidden=64, out_dim=1, p=0.2):
        super().__init__()
        self.lstm = nn.LSTM(in_dim, hidden, num_layers=2,
                            batch_first=True, dropout=p)
        self.head = nn.Linear(hidden, out_dim)
    def forward(self, x): # x: (batch, seq_len,
in_dim)
        out, _ = self.lstm(x)
        return self.head(out[:, -1, :]) # last timestep
```

Transformer (sequence/text)

```
# Use pretrained – almost never from scratch:
from transformers import AutoModel, AutoTokenizer
tok = AutoTokenizer.from_pretrained("bert-base-uncased")
backbone = AutoModel.from_pretrained("bert-base-uncased")
# Fine-tune head:
class Classifier(nn.Module):
    def __init__(self, backbone, K):
        super().__init__()
        self.b = backbone
        self.head = nn.Linear(backbone.config.hidden_size, K)
    def forward(self, ids, mask):
        out = self.b(input_ids=ids,
```

```
attention_mask=mask).last_hidden_state[:, 0]
return self.head(out)
```

▲ Canonical training loop

```
def train_epoch(model, dl, loss_fn, opt, device):
    model.train()
    tot, n = 0.0, 0
    for xb, yb in dl:
        xb, yb = xb.to(device), yb.to(device)
        opt.zero_grad(set_to_none=True) # faster than
zero_grad()
        pred = model(xb).squeeze(-1)
        loss = loss_fn(pred, yb)
        loss.backward()
        # Optional: clip gradient
        # torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
        opt.step()
        tot += loss.item() * xb.size(0); n += xb.size(0)
    return tot / n

@torch.no_grad()
def eval_epoch(model, dl, loss_fn, device):
    model.eval() # disables
Dropout, BN uses running stats
    tot, n = 0.0, 0
    for xb, yb in dl:
        xb, yb = xb.to(device), yb.to(device)
        pred = model(xb).squeeze(-1)
        tot += loss_fn(pred, yb).item() * xb.size(0); n +=
xb.size(0)
    return tot / n
```

Things that MUST be in every loop: opt.zero_grad() before backward;
model.train()/model.eval() toggles; with torch.no_grad() on eval;
move tensors to device.

▲ Optimizer + scheduler

Optimizer picker

Optimizer	When
Adam @ 1e-3	default for tabular / general
AdamW @ 1e-3, wd=1e-2	large NN / transformers (decoupled weight decay)
SGD + momentum 0.9	vision (with proper LR + nesterov)
RMSprop	RNN/LSTM (sometimes)

```
opt = torch.optim.AdamW(model.parameters(),
                        lr=1e-3, weight_decay=1e-2,
                        betas=(0.9, 0.999), eps=1e-8)
```

LR scheduler

```
from torch.optim.lr_scheduler import (
    ReduceLROnPlateau, CosineAnnealingLR, OneCycleLR)
```

```
# Reduce on plateau (simple, robust):
sched = ReduceLROnPlateau(opt, mode="min", factor=0.5,
patience=5)
sched.step(va_loss) # call AFTER eval
```

```
# Cosine annealing (transformer-style):
sched = CosineAnnealingLR(opt, T_max=epochs, eta_min=1e-6)
sched.step() # call once per epoch
```

```
# OneCycle (super-fast convergence; Smith 2018):
sched = OneCycleLR(opt, max_lr=1e-2, total_steps=epochs *
len(dl_tr))
# Call after EVERY batch.step()
```

Mixed precision (Ampere+ GPU)

```
# PyTorch 2.4+ canonical API (torch.cuda.amp.* deprecated):
DEV = device.type # "cuda" or "cpu" – works on either
scaler = torch.amp.GradScaler(DEV, enabled=(DEV=="cuda"))
for xb, yb in dl_tr:
    opt.zero_grad(set_to_none=True)
    with torch.autocast(device_type=DEV, dtype=torch.float16,
enabled=(DEV=="cuda")):
        pred = model(xb)
        loss = loss_fn(pred, yb)
        scaler.scale(loss).backward()
        scaler.step(opt)
        scaler.update()
# ~2x speed, ~50% memory; use bf16 on H100/A100 for numerical
stability
```

● Regularisation toolkit

Layer-level

Technique	Where
Dropout p=0.1-0.5	after activation; not before output
BatchNorm1d	between Linear + ReLU (tabular); after Conv (CNN)
LayerNorm	transformers / when batch dependence is bad
Weight decay	optimizer(weight_decay=1e-4) (L2)

Training-level

- **Early stopping:** monitor val loss; patience 10-30 epochs
- **Data augmentation:** images = torchvision.transforms; tabular = noise injection
- **Gradient clipping:** clip_grad_norm(p, 1.0) for RNN / unstable training
- **Label smoothing:** CrossEntropyLoss(label_smoothing=0.1) regularises classifiers
- **Mixup / Cutmix:** vision augmentation that boosts cal+acc

Early stopping (canonical)

```
best_val, patience, bad = float("inf"), 15, 0
for epoch in range(max_epochs):
    tr = train_epoch(model, dl_tr, loss_fn, opt, device)
    va = eval_epoch(model, dl_va, loss_fn, device)
    if va < best_val - 1e-4:
        best_val, bad = va, 0
        torch.save(model.state_dict(), "best.pt")
    else:
        bad += 1
        if bad >= patience: break
# Restore best:
model.load_state_dict(torch.load("best.pt"))
```

◆ Diagnose loss curves

Loss-curve signatures

Pattern	Diagnosis	Fix
Both flat-high	underfitting	bigger model / longer train / lower wd
Train ↓, val ↑ after ep N	overfitting at ep N	early stop / dropout ↑ / data ↑
Train >> val	data leak / val too small	re-check split + scaler fit
NaN loss after few steps	LR too high / numerical bug	LR / 10 / log scale / check loss API
Loss oscillates	LR too high / batch too small	LR ↓ / batch ↑ / smoothing
Train stuck at chance	label/scaler mismatch / dead ReLU	print pred shape, dtype, range

Sanity-check first batch

```
# Before training a full run:
xb, yb = next(iter(dl_tr))
print("x shape", xb.shape, "x mean/std", xb.mean().item(),
xb.std().item())
print("y shape", yb.shape, "y dtype", yb.dtype, "y range",
yb.min(), yb.max())
model(xb.to(device)).shape # output shape sanity
# Overfit a single batch – if you can't, the model/loss is wrong:
xb, yb = xb.to(device), yb.to(device)
for _ in range(200):
    opt.zero_grad(); l = loss_fn(model(xb).squeeze(-1), yb);
    l.backward(); opt.step()
print(l.item()) # should be ~0
```

Plot training history

```
import matplotlib.pyplot as plt
plt.plot(hist["tr"], label="train"); plt.plot(hist["va"],
label="val")
plt.yscale("log"); plt.legend(); plt.xlabel("epoch");
plt.ylabel("loss")
```

▲ Hyperparameter tuning

Optuna with PyTorch

```
import optuna
def objective(trial):
    hidden = trial.suggest_categorical("h", [32, 64, 128, 256])
    n_layer = trial.suggest_int("n_layer", 1, 4)
    p = trial.suggest_float("p", 0.0, 0.5)
    lr = trial.suggest_float("lr", 1e-4, 1e-2, log=True)
    wd = trial.suggest_float("wd", 1e-6, 1e-2, log=True)
    model = MLP(in_dim, hidden=tuple([hidden]*n_layer),
p=p).to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=lr,
weight_decay=wd)
    best_va = train_to_convergence(model, opt, dl_tr, dl_va)
    return best_va

study = optuna.create_study(direction="minimize",

pruner=optuna.pruners.MedianPruner())
study.optimize(objective, n_trials=50, n_jobs=1)
print(study.best_params)
```

Tuning priority order

1. **Learning rate** (log scale 1e-4 to 1e-2)
2. **Batch size** (power of 2; affects LR scaling)
3. **Architecture** (hidden width, depth)
4. **Dropout / weight decay** (regularisation strength)
5. **Scheduler** (ReduceLROnPlateau vs OneCycle)
6. **Optimizer** (Adam vs AdamW vs SGD)

LR finder (smith 2017)

```
# Quick LR sweep – increase LR exponentially each batch, find
where loss explodes
lrs, losses = [], []
opt = torch.optim.SGD(model.parameters(), lr=1e-7)
for i, (xb, yb) in enumerate(dl_tr):
    lr = 1e-7 * (10 ** (i / 30))
    for g in opt.param_groups: g["lr"] = lr
    opt.zero_grad(); l =
loss_fn(model(xb.to(device)).squeeze(-1), yb.to(device))
    l.backward(); opt.step()
    lrs.append(lr); losses.append(l.item())
    if l.item() > 4 * min(losses): break
# Plot lrs vs losses; pick LR one decade below the explosion
point
```

◆ Deployment

Save / Load (canonical)

```
# Save state dict, not the whole model (portable, smaller)
torch.save(model.state_dict(), "model.pt")
# Load (PyTorch 2.6+ defaults weights_only=True – safer):
model = MLP(in_dim, hidden=(128, 64), out_dim=1) # same
architecture!
model.load_state_dict(torch.load("model.pt", map_location="cpu",
weights_only=True))
model.eval()

# Full checkpoint (resume-on-crash):
ckpt = {
    "epoch": epoch,
    "model": model.state_dict(),
    "opt": opt.state_dict(),
    "scheduler": scheduler.state_dict(),
    "amp_scaler": scaler.state_dict() if 'scaler' in locals()
else None, # AMP only
    "best_val": best_val,
    "rng": {"py": random.getstate(), "np":
np.random.get_state(),
    "torch": torch.get_rng_state()},
    "arch_kwargs": {"in_dim": in_dim, "hidden": (128, 64),
"out_dim": 1},
    "torch_version": torch.__version__,
}
torch.save(ckpt, "ckpt.pt")
# Resume (weights_only=False because we have non-tensor state):
c = torch.load("ckpt.pt", map_location="cpu",
weights_only=False)
model = MLP(**c["arch_kwargs"]);
model.load_state_dict(c["model"])
opt.load_state_dict(c["opt"]);
scheduler.load_state_dict(c["scheduler"])
start_epoch = c["epoch"] + 1
# Save SKLEARN scaler SEPARATELY (cross-version safer than
embedding in torch.save):
import joblib; joblib.dump(sc, "scaler.joblib")

torch.compile for free 1.3-2x speedup (PT 2.x): model =
torch.compile(model) after instantiation.
```

TorchScript (production-ready)

```
model.eval()
# Trace (best for static graphs):
example = torch.randn(1, in_dim)
ts = torch.jit.trace(model, example)
ts.save("model_ts.pt")
# Or script (handles control flow):
ts = torch.jit.script(model)
# Load in C++/Java/iOS/Android – no Python needed.
```

ONNX export

```
torch.onnx.export(model, example, "model.onnx",
input_names=["x"], output_names=["y"],
dynamic_axes={"x": {0: "batch"}, "y": {0:
"batch"}},

opset_version=17)
# Serve with ONNXRuntime: 10-50x faster CPU inference.
```

FastAPI inference

```
from fastapi import FastAPI
from pydantic import BaseModel
import torch, joblib
app = FastAPI()
# Load the SAME ckpt dict + separate scaler.joblib produced
above:
ckpt = torch.load("ckpt.pt", map_location="cpu",
weights_only=False)
model = MLP(**ckpt["arch_kwargs"])
model.load_state_dict(ckpt["model"]); model.eval()
sc = joblib.load("scaler.joblib") # sklearn
StandardScaler
class Req(BaseModel): features: list[float]
@app.post("/predict")
```

```
def predict(r: Req):
    x = torch.tensor(sc.transform([r.features]),
dtype=torch.float32)
    with torch.no_grad():
        return {"y_hat": float(model(x).squeeze().item())}
```

Monitor

- **Input drift:** feature dist vs training (PSI / KS)
- **Prediction drift:** $\hat{y}$ distribution over time
- **Latency:** p50/p95/p99 per request
- **Re-train trigger:** PSI > 0.25 OR rolling metric degrades 10%

▲ Top traps (6-month memory)

- **Forgetting `opt.zero_grad()`** — gradients accumulate; training silently breaks.
- **Double sigmoid/softmax** — sigmoid + BCEWithLogitsLoss = double-applied. Use raw logits + BCEWithLogitsLoss.
- **Scaler fit on all data** — leakage. Fit on TRAIN only, transform val/test.
- **Peeking at test during tuning** — every peek poisons the held-out estimate. Use val for tuning.
- **Missing `model.eval()`** — Dropout still active + BN uses batch stats → noisy/wrong predictions.
- **Reporting in scaled units** — inverse-transform predictions before metric calls.
- **Judging imbalanced classifiers by accuracy** — 99% acc on 1% positives = "predict 0 always".
- **Dtype mismatch** — CrossEntropyLoss wants long; BCEWithLogitsLoss wants float. Mismatch silently fails or NaNs.
- **No deterministic seed for cuDNN** — set `torch.backends.cudnn.deterministic = True` for full reproducibility (small speed cost).
- **Not overfit-testing a single batch** — first sanity check; if you can't overfit one batch, the model/loss is wrong.

◆ Quick-reference (recall in 5 seconds)	
Task	Canonical pattern
Seed	<code>np.random.seed(42); torch.manual_seed(42)</code>
Device	<code>torch.device("cuda" if torch.cuda.is_available() else "cpu")</code>
Split	3-way: train (60%) / val (20%) / test (20%), stratify if cls
Scale	StandardScaler fit on TRAIN only
DataLoader	<code>shuffle=True</code> on train only; <code>batch_size</code> power of 2
Tabular model	MLP(2-3 layers, hidden 64-256, ReLU, Dropout 0.2)
Loss · binary	BCEWithLogitsLoss + raw linear output
Loss · multi	CrossEntropyLoss + raw linear, <code>y=long</code>
Loss · regression	MSELoss / L1Loss / HuberLoss
Optimizer	<code>AdamW(lr=1e-3, weight_decay=1e-4)</code>
Scheduler	<code>ReduceLR0nPlateau(patience=5, factor=0.5)</code>
Train step	<code>zero_grad</code> → forward → loss → backward → step
Eval step	<code>model.eval()</code> + <code>torch.no_grad()</code>
Early stop	patience 10-30 on val loss; restore best <code>state_dict</code>
Mixed precision	<code>autocast()</code> + <code>GradScaler()</code> (Ampere+)
Persist	bundle dict { <code>state_dict</code> , <code>scaler</code> , <code>arch_kwargs</code> , <code>cols</code> }